

11-768 HW2: Evaluating a Data-Visualization Agent

Olivia Brown

September 28th, 2026

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

The baseline validator’s predictions agree with the human labels barely above chance: macro-MCC is 0.0268, and every specific error score is near zero or negative. `wrong_data` and `hard_to_read` are particularly problematic, with the validator failing to identify any true positives. In all but `execution_failures`, all discrepancies are false negatives, meaning the validator passes as acceptable but shouldn’t be considered acceptable. On `execution failure`, the validator is particularly prone to false positives. Putting all of this together, this shows that the baseline tends to either call a run a total failure (FP in `execution_failure`) or call it acceptable (FN in `wrong_data`, `wrong_chart`, and `hard_to_read`). When a run produces a flawed figure, the validator barely ever names the specific defect, and when the run fails outright it frequently hallucinates a failure even though a valid figure exists. See below the exact error rates:

`execution_failure` MCC=-0.0022 TP=4 TN=65 FP=33 FN=8 support(+/-)=12/98 `wrong_-data` MCC=0.0000 TP=0 TN=73 FP=0 FN=25 support(+/-)=25/73 `wrong_chart` MCC=0.1096 TP=3 TN=27 FP=0 FN=68 support(+/-)=71/27 `hard_to_read` MCC=0.0000 TP=0 TN=59 FP=0 FN=39 support(+/-)=39/59 `macro_mcc` 0.0268

1.2 Validator Error Type 1

The first recurring error is random `execution_failure` predictions on runs where the agent succeeded. The validator predicts `execution_failure` on 37 runs, of which only 4 are true failures (33 false positives). Runs 004, 022, and 062 all exhibit the pattern. Each of these runs has a `valid_figure.png` and a trajectory ending in a normal submission, and the ground truth instead records content defects. For example, run 004’s axes-3 x-axis is linear where a log scale was requested, giving `wrong_chart` and `wrong_data`. In run 004 the model answers “YES” to the question “Did the agent crash, exhaust its turns, or give up without producing a valid figure?” and then narrates that “The agent successfully completed the task... The agent did not crash, exhaust its turns, or give up.” The baseline takes the leading token at face value and discards the rest as evidence, so the self-contradictory “YES” becomes a confident `execution_failure` error. Because `validate()` short-circuits as soon as the execution judge fires, the run is never checked for content defects at all, compounding the false positive with missed `wrong_data` and `wrong_chart` labels.

1.3 Validator Error Type 2

The second recurring error is systematic false negatives on the three content families: the judge sees a flawed figure and answers “NO” anyway. Runs 008, 025, and 026 illustrate it. In run 008

the value labels are hardly distinguishable from the background, yet the readability judge answers “NO” and praises the “distinct colors” and “properly configured” layout. In run 025 the request names a colour for each region, but every pie wedge is drawn in the same grey and the legend is empty. In run 026 the request asks for a sunburst with a hollow centre, but the figure shows solid circles. In all three cases the baseline predicted no errors at all. The mechanism is the judge’s single generic yes/no question: it asks “Is the rendered figure difficult or impossible to read?” (or “Does the plotted data differ...?” / “Does the figure fail to follow any part of the requested chart design?”) without ever using the concrete requirements from the task instructions, so the model describes the figure instead of comparing it against the spec, and defaults to the negative answer.

1.4 Proposed Validator Changes

To address Error Type 1, the validator should decide `execution_failure` without asking the model for a yes/no vote. Instead, it should check the facts directly. `Execution_failure` only happens when the run has no figure at all, or the trajectory ends in a crash, runs out of turns, or the agent gives up, so if a valid `figure.png` exists the run is not an execution failure. This removes the model from the decision, so a confused “YES” can no longer create a false positive. When the model is still asked for a judgment, it should explain its reasoning first and give a final verdict at the end, instead of answering yes or no upfront. To address Error Type 2, the validator should stop asking one broad question per family and instead list the specific requirements from the task instructions and check them one by one against the figure, the code the agent wrote, and the input data. A checklist makes the model compare each detail instead of describing the figure in general terms, so problems like run 008’s unreadable labels or run 025’s missing colors become failed checks instead of getting lost in an general description.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

The first change for Error Type 1 is a deterministic execution check: the validator calls `execution_failure` only when the run has no figure at all, or the figure file exists but cannot be opened as an image. This check is deterministic, with no model involved, using only the run’s figure path and the file on disk. Since the seed runs show that every true execution failure left no figure and every run with a figure was judged on content, a decodable figure is taken as proof the run executed, and the model is never asked to vote yes or no on it. The second change, targeting Error Type 2, is a checklist-style judgment for the three content families. Instead of the baseline’s three separate generic questions, the validator makes a single model call per run that asks the model to first list the concrete requirements from the task instructions, then check each requirement one by one against the figure, the plotting code in the trajectory, and the input data, marking each as met or violated with a short reason. Only after the checks does the model state its final decision, which the validator parses out of the response. This is a model-based check, but the structure forces point-by-point comparison instead of an overall impression. The evidence sent to the model is also curated rather than complete: the model receives the head of the trajectory (the task and opening actions) and its tail (the final plotting code and submission), with omissions marked, and the figure downscaled to 1024 pixels. We tested this choice directly against sending the complete trajectory, inputs, and full-resolution figure, and the complete version scored worse on every content family (macro-MCC 0.3998 versus 0.5026), because the long raw trajectories are mostly data dumps and

error tracebacks that bury the evidence that matters; the curation is an accuracy decision, not just a cost decision.

2.2 Seed-Set Performance vs. Baseline

The improved validator raises macro-MCC from 0.0268 to 0.5026 on the seed runs. The largest single gain is on `execution_failure`, which goes from an MCC of -0.0022 (4 TP, 33 FP, 8 FN) to a perfect 1.0 (12 TP, 98 TN, no errors of either kind): the deterministic check classifies every run correctly because the seed set exactly follows the rule that a run fails to execute if and only if it left no figure. `Wrong_chart` improves from 0.1096 to 0.5184 (3 TP to 49 TP of 71 positives), and `wrong_data` improves from 0.0 to 0.2649 (0 TP to 10 TP of 25), though both still miss about a third to half of their positives. `Hard_to_read` improves from 0.0 to 0.2271 but remains the weakest family (5 TP of 39 positives), and the first genuine false positives appear in `wrong_data` (11 FP vs. the baseline’s 0), a predictable cost of a judge that now actively looks for violations instead of defaulting to “no”.

`execution_failure` MCC=1.0000 TP=12 TN=98 FP=0 FN=0 support(+/-)=12/98 `wrong_data` MCC=0.2649 TP=10 TN=62 FP=11 FN=15 support(+/-)=25/73 `wrong_chart` MCC=0.5184 TP=49 TN=24 FP=3 FN=22 support(+/-)=71/27 `hard_to_read` MCC=0.2271 TP=5 TN=58 FN=34 FP=1 support(+/-)=39/59 `macro_mcc` 0.5026

2.3 Qualitative Analysis of Targeted Error Types

Both targeted error types are substantially mitigated. Error Type 1 (spurious `execution_failure`) is eliminated entirely: runs 004, 022, and 062, which the baseline wrongly called total failures, are now classified correctly, and the family scores a perfect MCC of 1.0 with zero false positives and zero false negatives. Run 004 in particular is now checked for content and both of its real defects are found (the axis scale and limits the instructions specified but the plot did not follow). Error Type 2 (content false negatives) is much reduced but not eliminated. Of the example runs, 008 and 026 now get content verdicts instead of a pass (026 gets both `wrong_data` and `wrong_chart` right), and 004 catches both `wrong_data` and `wrong_chart`. Run 008 shows the remaining gap: the validator now catches that orange flows were not filtered out (`wrong_data`), and its evidence matches the ground truth’s reasoning, but it still misses the near-white value labels that a human cannot read, so `hard_to_read` remains the weakest family (only 5 of 39 positives found). Run 025 shows another residual pattern: the checklist correctly calls the chart design wrong, but summarizes the figure as “a pie chart, not a sunburst chart” and stops there, missing the additional `wrong_data` (values not encoded) and `hard_to_read` (overprinted country codes) defects the human label records, so partial detection within a badly wrong figure is still incomplete. One regression appeared: a new class of `wrong_data` false positives (11 runs, e.g. 009 and 163), where the checklist flags an ordering or placement issue that the human label does not count as a data error (run 009’s real defects are a rotated-tick-label violation and overlapping tick labels, which the validator does also catch; run 163’s marked point is arguably misplaced, but the ground truth files it under `wrong_chart` only). This is the cost of a stricter judge.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

The improved validator judges every figure through a single checklist call: the model must first restate the requirements it can find in the task instructions, verify each one against the figure and the trajectory, and only then decide which of the three content families (if any) fired. Two classes of visualization task stress exactly the assumptions this design rests on, and I therefore picked one weakness class per assumption.

The first weakness class is "readability traps": figures whose instructions all hold (correct chart type, correct data, correct layout) but whose rendering defeats the reader. This could look like a legend stacked eight times on top of the data, tick labels that collide, value labels swamped by a background, and more. The checklist judges requirements one by one, and every individual requirement ("title X", "legend with entries A and B") can be met even while the composite image is unreadable, so a requirement-by-requirement judge has no natural slot for "the parts are all correct but the whole is unusable" and tends to pass such figures. The seed validator's weakest family was already `hard_to_read` (5 of 39 positives found), and I expected that gap to widen on tasks purpose-built so that the only defect is readability.

The second weakness class is "data transforms": figures where the drawn quantities are a formula over the input (normalization, filtering followed by an empirical distribution, log-scaled axes) rather than a column read straight from a file. Checking "does the plotted data match" then requires the validator to internally replay the transform - compute the filter, the cumulative fractions, or the percentage shares from the CSV - and compare the replayed numbers against pixel positions in the figure. A VLM judge cannot multiply, filter, or aggregate reliably, and nothing in my checklist tells it to try, so I expected systematic false negatives in `wrong_data` whenever the defect lives in the derivation instead of in a directly readable property. Both classes are also cheap for the agents to half-fail: a stacked-share chart plotted with raw counts instead of 100 percent shares is a two-line mistake that only a transform-aware reviewer can catch.

3.2 Task Design

I wrote five tasks, three in the readability-trap class and two in the data-transform class (the readability class got the extra task because its failure mode is cheaper to elicit across many chart types). All inputs are synthetic CSVs generated for this assignment (small hand-authored tables whose aggregate values are easy to verify by hand); each task directory under `tasks/` holds its `task.json` and its inputs.

- `dense-timeseries-legend` (`readability_traps`). Eight sensor lines (`sensors.csv`, 30 days x 8 columns) on a single axes that is deliberately crowded: eight near-parallel series, a legend that must go outside on the right, 30 rotated date tick labels. A reader can only attribute a line through the legend, so any legend placement mistake immediately hides data. Stress test: does the validator notice a legend that covers bars/lines or collides with tick labels, the seed validator's weakest spot.
- `contrast-heatmap` (`readability_traps`). A 6x6 correlation matrix (`correlation.csv`) as a coolwarm heatmap with the value printed inside every cell in black at font size 12 and x tick labels rotated 45 degrees. Stress test: text-over-content collisions (cell text vs. cell color both matter) and tick-label rotation, plus a colorbar requirement the checklist must remember to verify.
- `filtered-cdf-log` (`data_transforms`). From 500 latencies (`latencies.csv`), keep only measurements above 50 ms, compute the empirical CDF of the kept values, and draw it on a log x-axis with exact limits [50, 1000], a dashed reference line at 0.5, and a 'median' annotation at the crossing. Stress test: three chained transforms (filter, ECDF, log limits); the validator must notice an unfiltered or wrongly limited

curve. - grouped-barchart-values (readability_traps). Grouped bars from product_sales.csv with exact hex colors per quarter, a value label above every bar (no decimals), and a legend in the upper right that must not cover any bars - a corner case where the "correct" legend position sits over the tallest group's label region. - normalized-stacked-share (data_transforms). From raw respondent rows (platform_votes.csv), draw a horizontal 100 percent stacked bar chart of per-platform percentage shares, ordered youngest-at-top with the requested stack order and colors. Stress test: counting-to-share normalization is exactly the replay-a-formula check a non-computing judge fails; plotting raw counts yields visibly plausible-but-wrong bars.

3.3 Agent Runs and Labeling

I ran each task once per fixed agent (5 tasks for each of 3 agents) and reviewed every run against the task instructions, the trajectory, and the figure, writing the results into labels.json.

Agent-level results: GLM-4.7-Flash produced four clean figures and one flawed one; Ministral-3-14B produced three clean figures and two defective ones; Qwen2.5-Coder-3B produced no valid figure on four of the five tasks - it repeatedly burned its step budget trying to download nonexistent files or looping on errors, and on the one task where it did finish (normalized-stacked-share) the figure was wrong. The four execution failures were all Qwen runs, and every one of them left no figure.png with an empty submission, matching the assignment's terminal definition exactly.

There are validator failures and agent failures when executing these tasks. The failures that reflect the agents are (i) Qwen2.5-Coder-3B's inability to even reach a plotting step on four of five tasks - it invented nonexistent dependencies to download and looped on 404s or empty outputs until the step limit, an agent-harness failure that has nothing to do with visualization ability; (ii) GLM's legend bug (calling legend once per bar pair instead of once per chart) and Ministral's matshow default-tick slip, which are the kind of small API misunderstandings that any checklist reviewer could catch but the agents did not test their own output against; and (iii) Qwen's stacked-share figure, which quietly plotted raw proportions instead of shares - a silent wrong-data defect that produces a visually plausible chart. The failures that reflect my validator are the wrong_data false negatives on both transform tasks (the judge is never asked to replay a formula over the input) and the misfiled evidence on contrast-heatmap__ministral and normalized-stacked-share__qwen, discussed below.

The eight defective runs received these labels (one entry per family with evidence quoted from the run):

- contrast-heatmap__qwen, dense-timeseries-legend__qwen, filtered-cdf-log__qwen, grouped-barchart-values__qwen: execution_failure (no figure; loops on failed downloads/errors until the limit).
- contrast-heatmap__ministral: wrong_chart - numeric 0-5 tick labels on both axes instead of the required var1-var6 names, x ticks rotated almost 90 degrees instead of 45.
- grouped-barchart-values__glm: hard_to_read - the legend consists of eight repeated Q1/Q2 pairs and sits in the upper right covering the epsilon bars and their "250" label.
- grouped-barchart-values__ministral: wrong_data - product groups rendered in the order alpha, beta, delta, epsilon, gamma instead of the CSV sequence alpha, beta, gamma, delta, epsilon.
- normalized-stacked-share__qwen: wrong_data + wrong_chart + hard_to_read - raw proportions on a 0-0.14 axis instead of percentage shares summing to 100, entire platforms missing from some bars (55+ is a single red segment), stack order violated (35-44 leads with purple), single-entry legend in the top right covering the 18-24 bar.

The one ambiguous call was grouped-barchart-values__ministral: nothing in the instructions names an order for the product groups, and the assignment's "wrong order" clause is triggered by "a different order from the one requested". I labeled it wrong_data on the reading that the CSV row order is the requested sequence and a group-per-product task silently promises one row per

product in file order; a stricter reading would label the run clean. I flagged this in the disagreement analysis below.

3.4 Validator Performance on New Tasks

Running the Part 2 validator (output/pre_iteration_authored_predictions.json) against these labels gives:

```
execution_failure MCC=1.0000 TP=4 TN=11 FP=0 FN=0 support(+/-)=4/11 wrong_data
MCC=0.0000 TP=0 TN=9 FP=0 FN=2 support(+/-)=2/9 wrong_chart MCC=0.6236 TP=2
TN=7 FP=2 FN=0 support(+/-)=2/9 hard_to_read MCC=0.6708 TP=1 TN=9 FP=0 FN=1
support(+/-)=2/9 macro_mcc 0.5736
```

The deterministic execution check transfers perfectly: all four real failures found, no false alarms, because "no decodable figure.png" is exactly what all four runs have in common. The transform-dependent wrong_data score collapses to 0.0 (both positives missed), as predicted in the design section. The following are the validator's performance details on those new wrong_data tasks:

- normalized-stacked-share__qwen (three human labels, two validator labels). The validator's evidence - "x-axis values ranging from 0.00 to 0.14" - identifies the raw-count defect precisely but files it under wrong_chart instead of wrong_data; its color and legend complaints are real (wrong colors, wrong legend position and content). Same family, wrong evidence, as with the heatmap rotation case below: the judge sees the symptoms but not the mechanism, so it cannot name the family that a human would file the same observation under.
- contrast-heatmap__ministral: correct family (wrong_chart) but the evidence names "270-degree rotation" rather than the actual numeric tick labels - the judge latched onto a rotation cue at the figure's edge and mis-reported the defect's mechanism.

All four validator errors are either false positives (filtered-cdf-log__ministral: the validator claimed x limits start at "approximately 10" although the script calls plt.xlim(50, 1000) and the axis geometry confirms it; grouped-barchart-values__glm: flagged as wrong_chart "colors reversed" although the left bar is blue for Q1 as required - the actual defect is the legend, which the validator missed) or false negatives (grouped-barchart-values__ministral's ordering violation, and the missing wrong_data half of the qwen stacked run). Every missed or misfiled case involves either counting/deriving a quantity from the input or comparing pixel-level detail against the input file - the two things the checklist design never asks the model to do.

3.5 Harbor Packaging

All five tasks score 1.0 against the oracle agent.

The verifiers. Part 3 asks for a second way to grade each task: a fixed verifier that runs as part of the task itself and says whether a given solution passed. My verifiers work in three stages. First they check that the expected files are there: the figure exists and is a real, non-empty image, the input spreadsheet was not tampered with, and the plotting script is present. Second, they run the solution's plotting script again in a private copy of the workspace, and while it runs they record everything the script drew as structured data. Third, they compare that recorded data, plus a small JSON file with plotted values, against an answer key I computed when I designed the task and typed into the checker as fixed numbers.

The answer key is the important design choice. Because the expected values were computed by hand once, when the task was written, and are never recomputed inside the checker, a wrong solution cannot sneak past by making the same mistake the checker would make.

Writing these checkers caught me making assumptions too. My first draft of the stacked-bar checker assumed the row labels it saw were listed top-to-bottom (they were not) and that it could identify each bar’s color by position alone (three bars share each position). The reference solution failed its own checker until I fixed both assumptions.

Mutants. As a sanity test, I wrote mutants and checked that the verifier grades them as intended. One should fail; one should pass even though it is wrong, to expose what the checker is blind to.

- The first mutant (filtered-cdf-log) quietly skips one step of the task — it plots all 500 latency measurements instead of only the 260 above 50 ms. It looks plausible but every number it draws is wrong. It scores 0: the line has 500 points instead of 260, and its declared values (the count, the median, the checkpoint fractions) all disagree with the answer key. This confirms the cheap guarantee from the assignment: any mutant that changes a number fails, because the numbers were typed in by hand.
- The second mutant (normalized-stacked-share) gets every number and every structural detail right but sets every piece of text in the figure to a 2-point font, under three pixels tall. A person cannot read the chart, but the checker passes it (reward 1). Everything the checker can measure is correct. The fact that it is unreadable to a human reader is not something the programmatic checker can see.

What a fixed checker can and cannot grade. The four error families split cleanly at exactly the line the second mutant exposes.

- `execution_failure` (the agent crashed or produced no figure): easy for a checker. No image file, or an unopenable one, means failure. Both my checker and my validator get this family 100% right.
- `wrong_data` (wrong numbers or wrong ordering of the data): a checker can do it, but only because it holds the hand-derived answer key. The cost is that a human has to work out every expected value in advance, and the check is only as good as that work.
- `wrong_chart` (wrong chart type, colors, labels, layout): a checker can do it for the details my instructions pinned down exactly (type, colors, order, labels, scale, limits, legend placement), all of which are recorded facts about what was drawn. It cannot answer open-ended design questions like “was a grouped bar even the right chart for this data?” because there is no fixed answer to compare against. Every `wrong_chart` defect in my tasks was of the checkable kind.
- `hard_to_read`: not checkable. The second mutant is the proof: the checker knows the font size but cannot know whether the text is readable to a human looking at the picture. Some pieces can be approximated with geometry (my grouped-barchart checker does measure whether the legend’s box overlaps the bars), but the family as a whole is defined by how a reader experiences the image. This is the boundary where a human judge is genuinely needed, and it is why the rest of the assignment uses an AI model that looks at the picture.

Why this approach cannot grade the hidden test set. My checkers work only because I wrote the tasks they grade: I knew the data, so I could compute the answer keys, and I wrote the instructions, so I knew exactly which details to pin down. The hidden test set contains tasks I have never seen. Without an answer key, the only things left to check are content-free facts like “an image file exists,” and the mutants show what that is worth: the wrong-numbers mutant would slip straight through, and the unreadable one is invisible even to my full checker. A fixed checker also cannot be reused at all, since each one is customized to one specific. Grading unseen tasks therefore requires a judge with no answer key at all; one that reads the task’s own instructions, looks at the figure, and decides whether the figure does what the instructions asked.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

Part 3 exposed two kinds of failure my Part 2 validator could not have caught, and both had the same root cause: the judging model was never actually shown the program that drew the figure. It was told to check the plotting code, but the text excerpt it received had dropped the code entirely. So on the transform tasks it could not catch wrong numbers (nothing to trace), and on the readability tasks it guessed at pixel-level details (nothing to compare against).

The fix was to rebuild the validator’s judgment around the code. The validator now finds the plotting code mechanically. It takes the last command in the agent’s history that saved a figure and ran successfully, which recovers the exact code in every run that produced a figure. It then asks the judging model two focused questions instead of one broad one. The first question (“data audit”) is only about wrong data, and deliberately shows the model the code and the input data but not the finished figure, so it must trace the calculation rather than judge by how the picture looks. The second question (“checklist”) is about the other two families (wrong chart design and unreadable output) and shows the model both the code and the figure.

Testing against my own tasks then surfaced four specific mistakes by the judge, each of which became a rule:

- *Trust the code for what it says directly.* The judge twice contradicted the code in front of it: it claimed the x-axis limits were wrong when the code set them exactly as requested, and claimed two bar colors were swapped when the code assigned the exact requested colors. The instructions now say that whatever the code sets explicitly is a fact.
- *Do the arithmetic before flagging.* The judge flagged one solution for computing the distribution a slightly different way, but on this particular input the two methods give identical results (there are 260 kept values and all are distinct, so both methods land on exactly the same curve). The instructions now require the judge to actually work the numbers out and only report a data error if the result differs for at least one plotted value.
- *Compare orderings as written-out lists.* The judge missed a silent re-sorting bug: the solution’s code used a library function that quietly rearranges the categories into alphabetical order instead of the file’s original order. The instructions now require the judge to write both orders out, one under the other, and compare them line by line. The instructions also specify that if the task doesn’t state an order, the file’s own order is the expected one.
- *Check the labels under the labels.* The judge checked the chart’s axis labels and title but never the small tick labels (the values along each axis), missing a case where the ticks showed generic numbers instead of the required variable names. The instructions now say that whenever the task describes the input data’s structure, the tick-label contents are part of the requirements.

Two plumbing failures also turned up, both found by replaying the judge’s raw answers. First, truncation: the judge’s answer was cut off at the length limit before it reached its final verdict, and my code treated a missing verdict as “no error found.” Second, repetition: with randomness turned off (which keeps the judge’s answers stable), the judge would sometimes get stuck writing the same sentence over and over until it hit the length limit and treated the missing verdict as “no error found” again. That one silently lost seven problems the validator used to catch. The fixes: a longer answer limit, and if an answer still ends with no readable verdict, the validator asks the judge once more for just the verdicts.

I also tried, and abandoned, a separate pass where the judge looked at the figure alone and judged readability. It made things worse every time I tried it (the readability score on my tasks fell from 0.67 to 0.24 to 0.04), because the judge started hallucinating issues. Readability judgment stayed inside the checklist instead, as four fixed questions: is anything effectively invisible, does

any text overlap, is anything cut off by an edge, does the legend sit on top of data?

Finally, two findings from the seed runs shaped the finished instructions. The “trust the code” rule backfired on the very cases it was supposed to help: for a run whose defect was a series drawn in white on a white background, the judge read the requested color out of the code and declared it met. Questions about what is actually visible in the image are now exempt from the trust-the-code rule, and the judge is told to compare colors numerically (e.g. a color value of 1.0 on a white background) rather than by eye. And the first seed re-run produced 27 false “wrong data” alarms, every single one on a run that had a design issue and not a data issue. The audit now explicitly routes style problems to the `wrong_chart` family. That one change cut the false alarms to 19 without losing a single real catch.

4.2 Final Evaluation on Seed and New Tasks

MCC (Matthews correlation coefficient) scores each family from -1 to 1, where 1 is perfect judgment, 0 is no better than random guessing, and below 0 is worse than guessing.

The final validator (`output/final_seed_predictions.json`) results on the 110 seed runs:

`execution_failure` MCC=1.0000 TP=12 TN=98 FP=0 FN=0 support(+/-)=12/98 `wrong_data` MCC=0.3795 TP=17 TN=54 FP=19 FN=8 support(+/-)=25/73 `wrong_chart` MCC=0.6099 TP=51 TN=26 FP=1 FN=20 support(+/-)=71/27 `hard_to_read` MCC=0.3189 TP=8 TN=58 FP=1 FN=31 support(+/-)=39/59 `macro_mcc` 0.5771

For comparison, the pre-iteration Part 2 validator on the same runs scored `execution_failure` 1.0000, `wrong_data` 0.2649, `wrong_chart` 0.5184, `hard_to_read` 0.2271, `macro` 0.5026.

The final validator (`output/authored_predictions.json`) on the 15 self-authored runs:

`execution_failure` MCC=1.0000 TP=4 TN=11 FP=0 FN=0 support(+/-)=4/11 `wrong_data` MCC=0.3889 TP=1 TN=8 FP=1 FN=1 support(+/-)=2/9 `wrong_chart` MCC=1.0000 TP=2 TN=9 FP=0 FN=0 support(+/-)=2/9 `hard_to_read` MCC=0.6708 TP=1 TN=9 FP=0 FN=1 support(+/-)=2/9 `macro_mcc` 0.7649

For comparison, the pre-iteration Part 3 validator on the same runs scored `execution_failure` 1.0000, `wrong_data` 0.0000, `wrong_chart` 0.6236, `hard_to_read` 0.6708, `macro` 0.5736.

The final validator beats the pre-iteration one on every family on both sets: the seed-set average goes from 0.5026 to 0.5771, and the average on my own tasks goes from 0.5736 to 0.7649.

Getting there was not a straight line, and the steps backward were what taught me the most. On my own tasks, one intermediate version scored 0.3434 — the data audit worked, but the rest of the judge started inventing chart-design errors — and the next version, with the figure-only readability pass, fell to 0.4285 as the readability score collapsed to 0.04 under four invented problems. On the seed set, the first Part 4 run scored 0.4258, worse than the Part 2 validator I started from, because the repetition bug silently erased seven correct catches. Each regression was traced to its cause and fixed; the fixes, not the scores, are what carried into the final version. The overall lesson echoes Part 2: showing the judge the plotting code reliably helps on questions with a checkable answer (limits, colors, order, transforms), and hurts when it tempts the judge to skip looking at the actual picture on questions only the picture can answer (visibility, contrast, overlap). The final instructions hand the judge the code and the picture, and tell it explicitly which questions to answer from which.

4.3 Qualitative Analysis of Self-Authored Tasks

The final validator agrees with my human label exactly on 12 of the 15 runs. That includes all four crashes, all six heatmap and timeseries runs (the earlier color-swap, clipped-legend, and numeric-tick-label errors are all fixed), and the one run I labeled with all three content defects — the

validator found all three, and named the right evidence for each: the wrong data (proportions instead of shares), the wrong chart (legend position and stack order), and the unreadable output (legend covering a bar).

The three remaining disagreements are the interesting part:

- The validator flags a data error on `filtered-cdf-log_glm` that I say is clean. This is the run that computed the distribution by a slightly different method; the judge correctly noticed the method differs but then failed to finish the arithmetic showing the results are identical, and flagged it anyway.
- The validator clears `grouped-barchart-values_ministral` of a data error that I say is present. This is the alphabetical re-sort: the judge asserted the ordering matched without doing the written-out, line-by-line comparison the instructions demand, even though the solution’s code contains the library call that provably rearranges the order. These two runs together are the clearest picture of the judge model’s limit: it recognizes that a data question is being asked, but cannot reliably carry out the actual arithmetic.
- The validator misses the readability defect in `grouped-barchart-values_glm`, where the agent’s code drew a separate legend for every pair of bars, stacking eight duplicate legends on top of the data. The four readability questions include exactly this case (duplicate legend entries), yet the judge answers nothing for this run. This task’s defects survived every iteration, which makes it the single best measure of where this judge model’s ceiling is.

4.4 Further Changes

Four changes beyond the required one, each verified against the judge’s actual recorded answers. (1) The judge’s answer length limit went from 1600 to 3000 tokens, with a re-ask for just the verdicts when an answer ends without them. This fixed both the truncated-answer and the stuck-repeating failures, which had been silently turning caught defects into “no error found.” (2) If the data-audit answer cannot be parsed, the validator falls back to the checklist’s answer for `wrong_data` instead of assuming no error. (3) The readability questions now note that a legend placed outside the plot area cannot cover the data, which removes geometrically impossible complaints. (4) The audit was told that design problems belong to `wrong_chart`, not `wrong_data`, after the first seed re-run produced 27 false data alarms that were all really design defects. Change (4) had the largest measured effect: it cut seed `wrong_data` false positives from 27 to 19 without losing any true positives, raising the seed macro-MCC from 0.5588 to 0.5771. Changes (1) and (2) have no surviving examples left to measure on the final runs, but they were the direct fix for the seven runs the first re-run silently dropped. None of the four changes was tuned against the seed labels.